

Chase Robert Coleman

ASU ID: 1239783392

CSE 543: Software Security

Finding Crashes Assignment

June 16, 2026

Level 0: I inspected the source code and found that the program always called `puts((char*)1)`, which dereferenced an invalid memory address. Since the crash was hardcoded, any input successfully completed the level.

Level 1: The program read a single byte and treated it as a pointer when its value was 0x53 ('S'). Supplying the character S caused an invalid pointer dereference and crashed the program.

Level 2: The program checked for a magic integer value before casting user input to a function pointer. I supplied the correct signed integer representation of the magic value and used an invalid address to trigger a crash.

Level 3: Since source code was unavailable, I used `objdump` to analyze the binary. After identifying the hidden integer check and function pointer call, I passed the check and invoked an invalid address to crash the program.

Level 4: The program used `scanf("%s")` to read input into a 40 byte stack buffer without bounds checking. Supplying a very large input overflowed memory and caused the program to crash.

Level 5: The program copied user input into a 40 byte buffer using `strcpy()` without checking its size. A long input overflowed the stack buffer and triggered a crash.

Level 6: The vulnerable `memcpy()` was protected by a CRC check and only executed after satisfying it. Although the copy corrupted stack memory, the program immediately called `exit(1)`, preventing the corruption from causing a crash.

Level 7: I reverse engineered the binary using `objdump` and analyzed the custom CRC logic. After generating a payload that satisfied all checks, the out of bounds copy corrupted the stack and crashed the program.

Level 8: The packet parser performed arithmetic on the input length before validating it. By supplying malformed input, I triggered an out of bounds memory access that caused the program to crash.

Level 9: The program compared SHA256 digests using `strcmp()`, which is unsafe for binary data. Although this flaw could bypass equality checks, the final `memcpy()` did not overwrite control flow data, so no crashing input was identified.